GORGO: Online Tuning for Cross-Region Network-Aware LLM Serving

Anonymous Author(s)
Affiliation
Address
email

Abstract

In cross-region LLM serving, time-to-first-token (TTFT) is highly sensitive to per-request routing decisions. KV-cache reuse across replicas can eliminate much of the prefill computation on multi-turn workloads, but realizing this potential requires jointly balancing cache locality against replica load and network cost. Additionally, public datasets to evaluate these policies do not represent long-context, high prefix-reuse workloads: LMSYS and WildChat average 2 and 2.5 turns per conversation, 470 and 2,900 tokens per request, and 9 and 29% cross-user reuse, respectively. To capture the setting where a majority of requests contain reusable prefixes, we evaluate common routing policies such as least-load, prefix-aware, session-affinity, and others on traces from a production inference endpoint featuring multi-turn conversations ($\sim$82 requests/user), 7–45$\times$ longer prompts (21k avg tokens), and 2–6$\times$ higher KV-cache reuse (55%). On a multi-region GPU cluster whose cross-region network latency spans 26ms to 466ms across replicas, we test GORGO, a policy that jointly optimizes cache reuse, network latency, and queue depth, with weights tuned online to minimize p95 TTFT and no offline profiling. Against all other load-balancing policies, the GORGO family achieves the lowest TTFT at p50 and p95 in the tuning window and in every held-out evaluation window, and sweeps all reported percentiles under the heavier midday load, highlighting the impact of online joint optimization. The code is available at https://anonymous.4open.science/r/GORGO-D25A.

1 Introduction

Modern LLM chat services serve interactive workloads where perceived latency is dominated by time-to-first-token (TTFT). On a single replica, TTFT is the sum of three terms: (i) the network round-trip from the client proxy to the replica, (ii) queueing delay behind in-flight requests, and (iii) prefill computation for the input prompt. Inference engines such as SGLang [Zheng et al., 2024b] and vLLM [Kwon et al., 2023] expose radix-tree prefix caches that let an incoming request skip prefill for any token prefix already cached on the handling replica. When prompts are long and prefixes are widely reused, this saving dominates: a 20,000-token prompt that hits a 90% prefix match has only $\sim$2,000 tokens to prefill, reducing TTFT by an order of magnitude.

The KV-cache reuse opportunity is contingent on routing. If a request is sent to a replica that has not seen its prefix, the saving is forfeit. If many requests pile onto the same warm replica, that replica becomes a queueing bottleneck and the cache hit no longer pays for itself. If the closest replica with a warm cache is on the other side of the world, the network round-trip can swallow what the cache hit was supposed to save. The routing layer therefore has to balance three terms simultaneously: network distance to the replica, current load, and reusable cache state for the incoming prompt.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Standard heuristics such as least-load, prefix-aware, and session-affinity routing optimize one of these signals and ignore the others. For workloads with weak prefix reuse and uniform replica latencies, these heuristics are standard practice. However, interactive long-context traffic breaks both assumptions: production chat traces concentrate most tokens in a small set of returning users with substantial prefix overlap, and replicas placed in different regions have round-trip times that differ by an order of magnitude. The right routing target for a given request can change minute-to-minute as load shifts, and weighting the three terms with static rules leaves performance on the table.

Two further factors complicate empirical evaluation. First, the public chat datasets used in prior LLM-serving evaluations underrepresent long-context multi-turn traffic with reusable prefixes. LMSYS-Chat-1M [Zheng et al., 2024a] and WildChat-4.8M [Zhao et al., 2024] average 470 and 2,900 tokens per request, with intra-user prefix reuse below 10%. In order to understand the effects of routing as these factors scale, we evaluate on a production endpoint with 21,000 tokens per request on average and 53.7% intra-user prefix reuse—a regime where joint cache–network–queue trade-offs dominate TTFT. Second, evaluation methodology matters: a routing policy that wins p50 by herding traffic onto a single warm replica typically loses tail latency once that replica saturates. Reporting only one percentile hides this trade-off.

Contributions.

- A workload characterization (§2) placing three datasets on the same prefix-reuse and request-length axes. The production ARTChat-411k trace has 7–45× longer prompts and 2–6× higher token-weighted reuse than LMSYS-Chat-1M and WildChat-4.8M.
- A routing cost model and three variants collectively called GORGO (§3), which score each replica by an additive combination of network latency, estimated prefill cost on uncached tokens, and estimated decode cost on queued tokens. Variants differ in how scalar hyperparameters are set. The first variant is a static configuration (`gorgo-static`), the second is a per-replica online fit (`gorgo-autotune`), and the third is a Gaussian (1+1) evolution strategy hill climb that directly minimizes p95 TTFT (`gorgo-hillclimb`).
- An experimental setup (§4) running each of eight policies on its own dedicated three-replica fleet of L40S SGLang servers in Asia, Europe, and the US East coast. Policies are benchmarked in parallel to normalize the variability of cross-region RTT, which spans 26–466 ms.
- Empirical results (§5) across three 30-minute production-trace windows (nighttime tuning W1, nighttime held-out W2a, midday held-out W2b). The GORGO family achieves the lowest TTFT p50 and p95 in all three windows against five baseline policies, and sweeps all three percentiles in the midday window; nighttime p99 differences sit at the noise floor discussed in §6. *[Reviewer comment: The previous wording ("lowest TTFT at every percentile in all three windows") is contradicted by the paper's own tables: `random` has the best W1 p99 (Table 2) and `least-request` the best W2a p99 (Table 2). Claim narrowed to match the data — a reviewer checking the tables against the abstract would flag this immediately.]*

2 Workload Characterization

Routing policies that perform well on short, low-reuse prompts may perform differently on long, high-reuse prompts because the prefill term in TTFT scales with uncached input tokens. We measure three datasets along the same axes: request length, conversational structure, and token-weighted prefix reuse.

Datasets. **LMSYS-Chat-1M** [Zheng et al., 2024a] is a dataset of one million chatbot conversations from a publicly hosted demo of a Vicuna model and the ChatbotArena website. **WildChat-4.8M** [Zhao et al., 2024] is a corpus of 3.2M ChatGPT-proxy conversations grouped by client IP. Our **ARTChat-411k** dataset contains chat completion requests from a production GLM 5.1 model endpoint hosted by an industry partner [GLM Team, 2024]. It contains 411,169 requests across 4,984 distinct users, yielding ∼82 requests per user on average. We used the request metadata for real-world timestamps and tokenized request bodies for reuse calculation.

Table 1: Workload characterization. Token counts use the same byte-pair tokenizer in all rows. Reuse percentages are token-weighted prefix-trie savings as described in §2.

Dataset	Total reqs	Users	Avg input tokens	Intra-user reuse	Global reuse
LMSYS-Chat-1M	1,000,000	—	467	—	9.0%
WildChat-4.8M	3,199,860	1,833,730	2,925	5.3%	34.4%
ARTChat-411k production	411,169	4,984	21,043	53.7%	55.3%

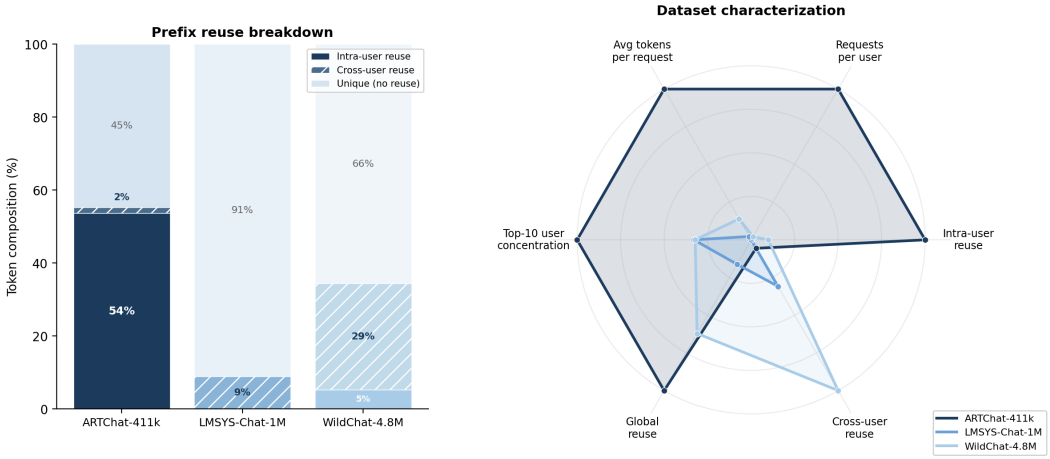

Figure 1: *Left*: Token composition per dataset. ARTChat-411k's reuse is overwhelmingly intra-user (54%), reflecting multi-turn dialogue; WildChat's is predominantly cross-user (29%, shared templates); LMSYS has minimal reuse (9%, all cross-user). *Right*: Radar chart with six axes normalized to the dataset maximum. ARTChat-411k dominates on every axis relevant to routing: long prompts, high multi-turn density, concentrated users, and strong intra-user prefix reuse.

We measure prefix reuse with a token-weighted prefix trie that ingests each request's tokenized message sequence in arrival order, partitioned by user where the dataset carries user identity. The trie reports *intra-user savings* (fraction of tokens matching a prefix from the same user's prior requests), *cross-user savings* (additional fraction from pooling across users), and *global savings* (sum). LMSYS does not carry user identity, so we report only the global figure.

Three contrasts stand out (Table 1, Figure 1). *Request length*: ARTChat-411k prompts are 45× longer than LMSYS and 7× longer than WildChat. At typical rates, prefill on a 21,000-token uncached prompt takes on the order of seconds.

User concentration: ARTChat-411k has 82 requests per user on average; WildChat has 1.7; LMSYS does not carry user identifiers, so intra-user statistics cannot be computed. Intra-user reuse depends on returning users, which the public sets simply do not have. *Reuse magnitude*: 53.7% of ARTChat-411k tokens are part of a prefix seen earlier from the same user (5.3% for WildChat, essentially zero for LMSYS). The bulk of WildChat's 34.4% global reuse comes from cross-user shared structure (chat templates, viral prompts), not sustained per-user dialogue. Thus, for routing, LMSYS and WildChat live in a regime where prefix-aware policies have little to win. On ARTChat-411k, a strategy that lands a user on a replica that has seen their conversation can save more than half of the prefill.

Experimental windows. We focus on two consecutive 30-minute windows taken from ARTChat-411k traffic on April 2nd, 2026: 00:30–01:00 UTC (W1, tuning) and 01:00–01:30 UTC (W2a, held-out evaluation). A third window at 12:30–13:00 UTC on April 2nd, 2026 (W2b, midday diurnal evaluation) tests generalization to heavier daytime load. Two additional stress-load windows (W3 & W4, run at higher sustained load with the same c=64 concurrency) appear in Appendix D and inform the live-tuning instability discussion in §6. Traces are replayed at original arrival speed and pre-filtered to 24,000 input tokens. Each window contains approximately 4,800 (W1), 4,200 (W2a), and 3,100 (W2b) arriving requests; each policy independently processes the full trace, so

the per-policy n in result tables reflects requests that completed within the window at the per-policy concurrency limit. *[Reviewer comment: These arrival counts conflict with later statements. W2b is described as "heavier daytime load" with "47% more requests" (Table 2 caption) and $\sim$1.7 req/s midday vs. $\sim$1.2 req/s nighttime (Appendix C), yet 3,100 W2b arrivals would make it the* smallest *window. The rate figures are consistent with the completed counts ($1.2 \times 1800 \approx 2{,}160 \approx n_{\mathrm{W1}}$; $1.7 \times 1800 \approx 3{,}060 \approx n_{\mathrm{W2b}}$), which suggests 4,800/4,200 are pre-filter totals rather than replayed arrivals — please verify against the run manifests and state which it is. Separately, explain why n is identical across policies within a window despite per-policy completion; if results are truncated to a common completed prefix, say so.]* Output is capped at 128 tokens to prevent decode from dominating latency.

3 GORGO

[Reviewer comment: The acronym GORGO is never expanded anywhere in the paper. Reviewers reliably ask; either expand it at first use or state that it is a codename.]

3.1 TTFT Decomposition

Let a request r with token sequence x_r be routed to replica u. Its TTFT decomposes as

$$\mathrm{TTFT}(r,u) = \mathrm{rtt}(u) + w_{\mathrm{queue}}(u,t_r) + t_{\mathrm{prefill}}(u) \cdot |x_r \setminus c_u(t_r)| + \varepsilon, \tag{1}$$

where $\mathrm{rtt}(u)$ is the round-trip from the routing proxy to replica u; $w_{\mathrm{queue}}(u,t_r)$ is queueing delay; $t_{\mathrm{prefill}}(u)$ is the per-token prefill rate; $c_u(t_r)$ is the set of token sequences cached on u at arrival time t_r, so $|x_r \setminus c_u(t_r)|$ is the number of uncached tokens; and ε collects scheduler overhead and batching variance. Cross-region $\mathrm{rtt}(u)$ ranges from 26 ms to 466 ms in our cluster; the prefill term can be the largest and most routing-sensitive term at typical rates of 0.1–1 ms per uncached token on a 21,000-token prompt.

3.2 Cost Model

A GORGO replica score is a weighted sum of the three controllable terms in Equation 1:

$$\mathrm{score}(u,r) = w_{\mathrm{rtt}}\cdot\mathrm{rtt}_u + w_{\mathrm{prefill}}\cdot t_{\mathrm{prefill}}\cdot(|x_r| - |x_r \cap c_u|) + w_{\mathrm{load}}\cdot\alpha\cdot(\mathrm{queued\_tokens}_u + \mathrm{used\_kv\_tokens}_u). \tag{2}$$

The proxy sends r to the replica with the minimum score. Three signals feed the score: (i) rtt_u from the Exponential Weighted Moving Average (EWMA) of a lightweight network probe decoupled from the metrics scrape; (ii) $|x_r \cap c_u|$ from the proxy's local radix trie of cached prefixes per replica, refreshed on every request completion; and (iii) load from the proxy's in-flight token counter combined with SGLang's reported KV-cache occupancy. The score has two physical rate parameters—per-token prefill rate t_{prefill} and per-token load coefficient α—and three dimensionless weights $w_{\mathrm{rtt}}, w_{\mathrm{prefill}}, w_{\mathrm{load}}$ that scale each term. The three variants below differ in how these five scalars are set.

3.3 Three Variants of Weight Selection

gorgo-static fixes all five scalars before the run and never adjusts them. In W1 we use manually chosen starter values (w_{rtt}=1, w_{prefill}=1, w_{load}=1, t_{prefill}=0.07, α=0.06). In W2, `gorgo-static` deploys the weights learned by `gorgo-hillclimb` at the end of W1 ($w_{\mathrm{rtt}}\approx$0.39, $w_{\mathrm{prefill}}\approx$1.88, $w_{\mathrm{load}}\approx$6.38, with t_{prefill} and α held at their W1 baseline values) and never adjusts them again. This isolates the contribution of the cost model independent of online adaptation.

gorgo-autotune holds the dimensionless weights fixed at w_{rtt}=w_{prefill}=w_{load}=1 and re-fits the two physical rate parameters every 16 new request samples from the trailing 64-sample window, partitioned by destination replica. t_{prefill} is set to the median observed prefill rate $(\mathrm{TTFT} - \mathrm{rtt})/|x_r \setminus c_u(t_r)|$; α is set to the median observed decode rate $(\mathrm{total\ time} - \mathrm{TTFT})/n_{\mathrm{output}}$, since queued and in-use tokens drain at the decode rate. It adapts to per-replica hardware drift and RTT shifts, but treats the physical rates as identifiable quantities rather than black-box knobs. Results for `gorgo-autotune` on the main production trace are omitted from main tables due to a rate-inversion instability: when prefix hit rate is high, the uncached-token count $|x_r \setminus c_u(t_r)|$ approaches zero, causing the median-of-rates estimator to produce inflated t_{prefill} estimates that effectively blacklist the

affected replica. The resulting routing depends on which replica's rate inverted rather than on load or cache state, making results unreliable as a policy evaluation. Behavior on a higher-throughput short-prompt workload appears in Appendix A; stress-load results appear in Appendix D.

gorgo-hillclimb holds t_{prefill} and α at their W1 baseline values and runs a Gaussian $(1{+}1)$-evolution strategy [Rechenberg, 1973] over the three dimensionless weights $(w_{\text{rtt}}, w_{\text{prefill}}, w_{\text{load}})$ in log-space. The objective is the negative p95 TTFT of the rolling 64-request window. Every 16 new samples the tuner scores the current weights, accepts or rejects the previous candidate against the incumbent, and proposes a new candidate by perturbing each weight as $\exp(\log(v) + \sigma\,\mathcal{N}(0,1))$, with initial step size $\sigma_0{=}0.5$ and σ adapted by Rechenberg's 1/5-success rule. *[Reviewer comment: σ_0 previously appeared only in the NeurIPS checklist; surfaced here for reproducibility. A duplicate mention of the 1/5-success rule in the same sentence pair was removed.]* This variant treats the dimensionless weights as black-box knobs: it finds whatever values produce the best p95 TTFT under the current arrival distribution, with no offline profiling. The two adaptive variants are complementary rather than the same method at different granularity: `gorgo-autotune` fits the two *physical* rates *per replica* by median-of-rates regression with the weights frozen at 1, whereas `gorgo-hillclimb` searches the three *dimensionless* weights *globally* (one shared vector across replicas) by evolution strategy with the rates frozen. They adapt disjoint parameter subsets by different mechanisms; in particular `gorgo-autotune` is not a per-replica `gorgo-hillclimb`.

4 Experimental Setup

Hardware and software. Each policy runs on its own dedicated three-replica fleet so routing decisions cannot warm or cool another policy's caches. Each fleet consists of one 2×L40S SGLang [Zheng et al., 2024b] server (tensor-parallel 2, 96 GB VRAM per replica) in each of Asia (Seoul), Europe (Frankfurt), and US East (Ashburn), serving Qwen3.5-35B-A3B-FP8 [Qwen Team, 2025]. The model's native context window is 32,768 tokens; we cap workload input at 24,000 tokens to leave KV headroom for in-flight batches. A CPU proxy in US East (Ashburn) hosts the routing policy and replays the trace.

The proxy scrapes Prometheus metrics from each replica every ∼30 s and runs a separate lightweight HTTP probe to estimate per-replica RTT, smoothed with an EWMA. The probe is decoupled from the metrics scrape to avoid polluting the RTT estimate with SGLang handler contention. Before each run, a homogeneity check issues 16 streaming requests directly to each replica, bypassing the proxy; runs with worst-to-median TTFT ratio exceeding 3× are flagged. No such run occurred in these experiments.

Cross-region RTT distribution. Probed from the US East proxy, EWMA-smoothed RTT to the US East replica is 26–40 ms, to Frankfurt is 90–130 ms, and to Seoul is 260–466 ms. The Seoul upper bound reflects routing-table churn during the trace. The overall range of 26–466 ms represents an 18× spread. Figure 2 shows the full RTT time series over the W1 window.

Baselines, policies, and windows. We compare the three GORGO variants of §3 (`gorgo-static`, `gorgo-autotune`, `gorgo-hillclimb`) against five baseline policies. `random` samples a replica uniformly per request and serves as the lower bound. `least-request` routes to the replica with the fewest in-flight requests, taking the larger of the proxy's in-flight view and SGLang's scraped running-request count to bridge the ∼10 s staleness between metrics scrapes. `least-load` routes to the replica with the lowest aggregate token-weighted load, summing running and queued requests with used and queued KV tokens; it is heavier-signal than least-request but cache-blind. `prefix-cache` is the longest-prefix-match policy of Preble [Srivatsa et al., 2025] as packaged in AIBrix [AIBrix Team, 2024]: it picks the replica with the longest cached prefix match and falls back to least-request when the running-request imbalance exceeds a soft threshold. `simple-session-affinity` hashes the first 256 token IDs of the prompt modulo the number of replicas, maximizing intra-user reuse but providing no load-escape mechanism. We run all eight policies in parallel, each on its own dedicated fleet, simultaneously starting at the same wall-clock time with the same input trace. `gorgo-autotune` results for ARTChat-411k are omitted from main tables due to a rate-inversion instability at high prefix hit rates (see §6); its behavior on a high-throughput workload appears in Appendix A. W1 supplies `gorgo-hillclimb` with uniform starting weights (rtt_weight=1.0, prefill_weight=1.0, load_weight=1.0); W2 uses

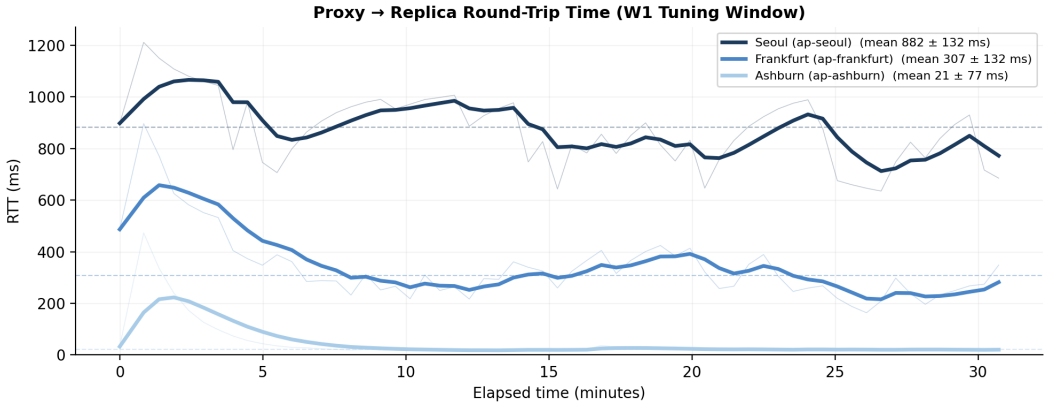

Figure 2: Proxy-to-replica RTT over the W1 tuning window. Bold lines show the EWMA-smoothed signal (α=0.3) that GORGO uses for routing; faint lines show raw probe samples. Dashed lines show per-replica means. Seoul exhibits periodic spikes from routing-table churn; Frankfurt is moderately variable; Ashburn is near-constant. The 18× spread motivates network-aware routing.

the weights that `gorgo-hillclimb` learned during W1 (rtt_weight≈0.39, prefill_weight≈1.88, load_weight≈6.38). `gorgo-static` in W2 deploys those weights without further adjustment. Concurrency is 64 in-flight requests per proxy. *[Reviewer comment: Eight policies are said to run in parallel, but the main tables list six: `gorgo-autotune`'s omission is documented, while `gorgo-static` — which per §3 ran W1 with manual starter weights — is absent from Table 2 without explanation. Either add the row or state why it is excluded; as written, a reviewer cannot tell whether the starter weights performed poorly.]*

Metrics. For each policy and window we report TTFT p50, p95, and p99 over all successful streaming responses. Traces are pre-filtered to the model's effective context boundary; all policies achieve 100% success rate in the three main windows (W1, W2a, W2b); the stress windows of Appendix D see ∼10% timeouts.

5 Results

5.1 p95 TTFT Objective

Tuning window. Across all three windows GORGO posts the lowest TTFT p50 and p95 (Table 2, Figure 4): `gorgo-hillclimb` during tuning (W1) and `gorgo-static` on the held-out W2a/W2b. The margin over the strongest baseline *widens with load* — from ∼9% on the held-out nighttime window to ∼12% at midday — because the learned w_{load}=6.38 rebalances traffic that hash-based session-affinity cannot: `gorgo-static`'s p95 degrades only 27% night→midday while session-affinity degrades 55%. On both held-out windows GORGO also sweeps E2E. The one place a baseline leads is p99 *during tuning* (an ES exploration tax, 2.66 s vs. `random`'s 2.00 s); this tail cost vanishes once `gorgo-static` deploys frozen weights, and the nighttime p99 spreads among the top policies are within one standard error (§6).

Starting from rtt_weight=1.0, prefill_weight=1.0, load_weight=1.0, the ES converged to rtt_weight≈0.39, prefill_weight≈1.88, load_weight≈6.38 over the 30-minute window. The high load_weight means the ES discovered that actively avoiding loaded replicas is critical at c=64 concurrency, even at the cost of occasionally routing to a cache-cold replica.

5.2 p50 TTFT Objective

To test whether the learned weights depend on the objective percentile, we re-run W1 tuning with a p50 TTFT objective (same trace, same hyperparameter ranges).

The p50 objective discovers a qualitatively different operating point (w_{rtt}=1.21, w_{prefill}=0.52, w_{load}=1.69; Table 4). With it GORGO again wins TTFT p50 and p95 in all three windows (Table 3),

Table 2: **p95-objective TTFT/E2E across all three windows** (ARTChat-411k, April 2nd 2026; seconds). GORGO is `gorgo-hillclimb` in W1 (tuning) and `gorgo-static` (frozen W1 weights) in the held-out W2a/W2b. Bold = best in column within each window. All policies 100% success; n=2,095 (W1), 2,207 (W2a), 3,071 (W2b). ITL (8–12 ms) and decode throughput (96–127 tok/s) varied little across policies and are omitted.

	TTFT (s)			E2E (s)		
Policy	p50	p95	p99	p50	p95	p99
W1 — tuning window, 00:30–01:00 UTC (nighttime)						
GORGO (hillclimb)	**0.180**	**1.010**	2.660	1.42	**2.14**	2.95
simple-session-affinity	0.194	1.185	6.757	1.48	2.98	3.47
least-request	0.197	1.179	2.062	**1.24**	2.24	3.07
least-load	0.271	2.158	8.750	1.40	3.30	**2.92**
prefix-cache	0.283	1.305	2.210	1.31	2.74	3.32
random	0.292	1.192	**1.996**	1.27	2.38	3.24
W2a — held-out, 01:00–01:30 UTC (nighttime)						
GORGO (static)	**0.186**	**0.896**	1.844	**1.23**	**1.99**	**2.89**
prefix-cache	0.190	1.041	2.011	1.35	2.54	3.16
simple-session-affinity	0.201	0.981	1.785	1.54	2.64	3.47
least-request	0.260	1.131	**1.758**	**1.23**	2.16	2.90
least-load	0.276	1.144	1.808	1.42	2.43	3.28
random	0.292	1.280	2.481	1.30	2.49	4.38
W2b — held-out, 12:30–13:00 UTC (midday diurnal)						
GORGO (static)	**0.195**	**1.136**	**2.060**	**1.29**	**2.46**	**3.24**
simple-session-affinity	0.234	1.519	2.539	1.69	3.78	6.70
prefix-cache	0.280	1.558	2.491	1.55	3.59	7.14
random	0.292	1.518	2.457	1.37	3.01	4.53
least-request	0.294	1.297	2.064	1.36	2.60	3.48
least-load	0.297	2.030	3.765	1.73	4.00	5.99

and the median-TTFT margin is in fact *larger* than under the p95 objective (e.g. 21.7% vs. 16.4% at midday). But the RTT-first routing concentrates traffic on the nearest replica and pays a consistent E2E-p95 penalty (∼6–9% behind `least-request`) — the mirror image of the p95 policy, which wins both TTFT and E2E. The lower w_{load}=1.69 (vs. 6.38) is what buys the sharper median at the cost of tail load-balancing.

5.3 Objective Sensitivity: Weight Comparison

Table 4 compares the learned weights across objectives.

The p95 objective drives the policy toward cache-first routing (w_{prefill}=1.88) with aggressive load avoidance (w_{load}=6.38), because tail requests are the ones stuck behind queues on cache-cold replicas. The p50 objective drives toward RTT-first routing (w_{rtt}=1.21, 3.1× larger), because the typical request is short enough that network latency dominates over cache effects. The p50-tuned policy achieves 0.163 s p50 TTFT (vs. 0.180 s for p95-tuned), a 9% improvement on the metric it optimizes.

5.4 When does GORGO win?

The results above share one mechanism. GORGO's TTFT advantage does not come from a higher cache hit rate: `simple-session-affinity` achieves the *highest* hit rate of any policy (Figure 5, left) yet posts among the worst p95, because hashing pins every session to one replica and cannot escape the resulting load. GORGO instead keeps *most* of that cache locality while staying balanced — it concentrates only moderately on cache-warm replicas (Figure 5, right) and prices the queueing that prefix-only routing ignores. That is the "why": on a loaded, high-reuse fleet the binding constraint is queueing on the cache-warm replica, and GORGO is the only policy that sees the cache term and the load term at once.

This also bounds *where* the advantage holds. GORGO needs both real prefix reuse and real load to exploit, and ARTChat-411k supplies both (Figure 1). On short-prompt, low-reuse public traffic

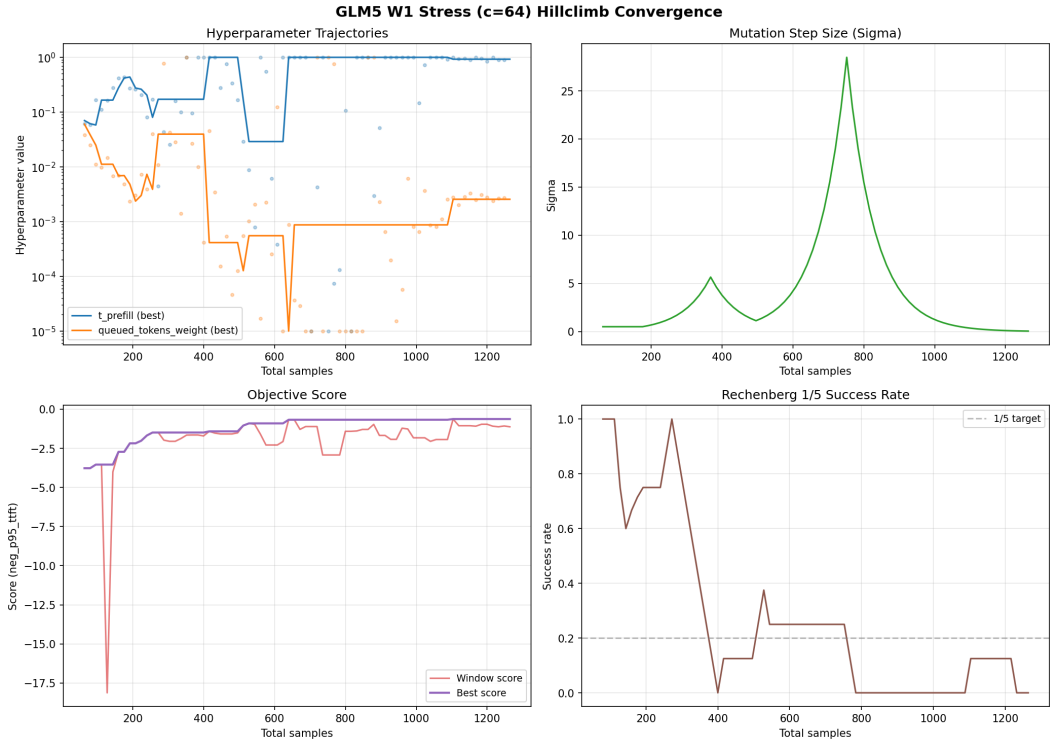

Figure 3: (1+1)-ES convergence over the W1 tuning window (p95 objective). *Top-left*: weight trajectories—w_{load} (orange) climbs to 6.38 while w_{rtt} (green) settles at 0.39; dots show ES proposals. *Top-right*: step size σ peaks then decays as the ES converges. *Bottom-left*: best p95 TTFT improves from 3.0 s to 0.5 s. *Bottom-right*: acceptance rate drops toward the 1/5 target.

the cache lever nearly vanishes and GORGO's gains collapse toward `random`: we show this directly on a WildChat replay in Appendix A, and the public-dataset comparison of §2 (LMSYS, WildChat) quantifies how far that regime sits from production traffic. The honest reading is therefore not that GORGO dominates everywhere, but that *where reuse and load co-occur* — the production regime practitioners actually pay for — jointly cost-aware routing beats every single-signal baseline, and degrades gracefully to baseline where they do not.

6 Limitations

p99 noise floor. Each window has ≈ 2,100–2,200 successful responses, so the p99 standard error is $\sigma_{\text{TTFT}}/\sqrt{n \cdot 0.01 \cdot 0.99} \approx 0.07$ s. In W2a the p99 ordering in fact favors `least-request` (1.758 s) over `gorgo-static` (1.844 s) by 0.086 s, and in W2b `gorgo-static`'s p99 margin over `least-request` is 0.004 s (2.060 vs. 2.064 s); both gaps are within roughly one SE. Tail-percentile differences between the top policies are therefore not individually significant at this sample size, in either direction; the p50 and p95 results carry the comparison. *[Reviewer comment: The previous text quoted p99 values (1.626 s and 1.702 s) that appear in no table in the paper, and claimed a GORGO p99 "win" that Table 2 does not show. Rewritten from the published table values — please trace where the stale numbers came from (an earlier run?) and confirm the tables are the canonical ones.]*

gorgo-autotune rate-inversion instability. The prefill-rate estimator sets $\hat{t}^{(r)}_{\text{prefill}} = \text{median}\{(\text{TTFT}_i - \text{rtt}_i)/|x_r \setminus c_u(t_r)|\}$ over the trailing window. When prefix hit rate is high—as it is on the ARTChat-411k production trace (∼55% reuse)—the denominator $|x_r \setminus c_u(t_r)|$ is small for most requests, and small timing noise in TTFT inflates the estimate by orders of magnitude. In experiments, the Seoul replica reached $\hat{t}_{\text{prefill}} = 77$ ms/tok versus a calibrated value

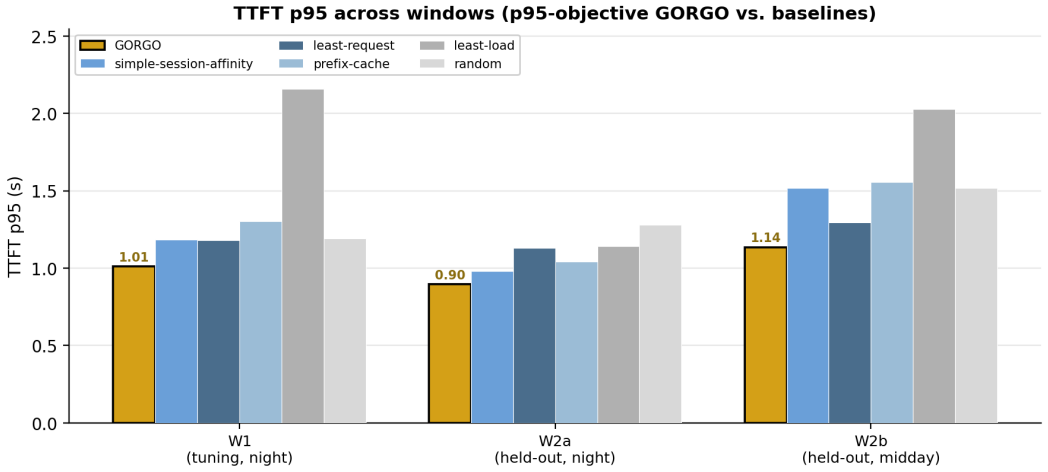

Figure 4: TTFT p95 across the three windows, GORGO (gold) vs. baselines (p95 objective). GORGO is lowest in every window, and its margin over the best baseline widens from the held-out nighttime window (W2a) to the heavier midday window (W2b). Full p50/p95/p99 and E2E values are in Table 2.

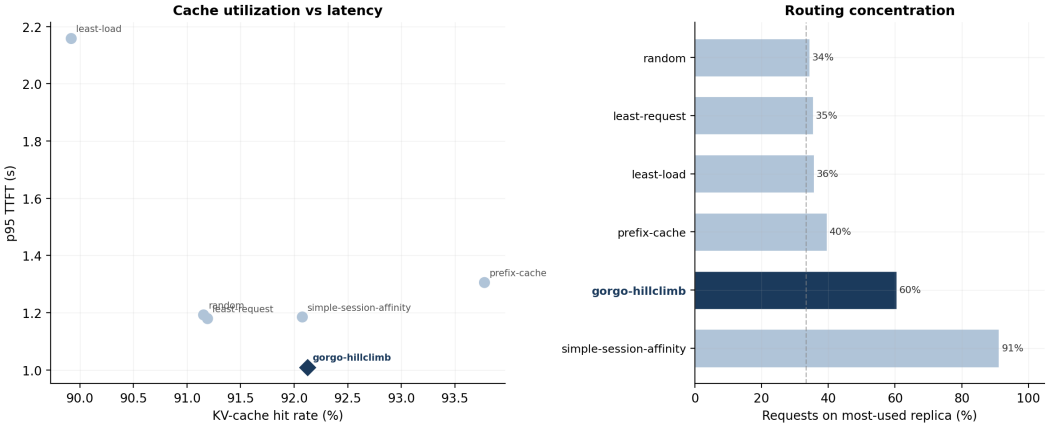

Figure 5: *Left*: KV-cache hit rate vs. p95 TTFT on W2a. *[Reviewer comment: "W2" is ambiguous now that W2a and W2b are distinguished — confirm which window this figure plots; W2a assumed.]* Bottom-right is ideal (high cache, low latency). `gorgo-static` and `gorgo-hillclimb` achieve both; `simple-session-affinity` achieves the highest cache hit rate but the worst p95 because it cannot rebalance load. *Right*: routing concentration (share of requests on the most-used replica). The dashed line marks uniform (33%). GORGO variants concentrate moderately on cache-warm replicas without the pathological imbalance of `simple-session-affinity`.

of $\sim 0.1\,\mathrm{ms/tok}$, effectively blacklisting it. Routing then reflects which replica's rate inverted, not load or cache state—an artifact, not a policy signal. `gorgo-autotune` is therefore omitted from the main production-trace tables. The ES avoids this by treating the dimensionless weights as black-box knobs rather than inverting the TTFT equation. Future work could address the instability by filtering near-zero denominators, using a regularized minimum, or estimating rates in a low-reuse warm-up period before switching to the full trace.

Scope of fleet. Each policy runs on its own dedicated three-replica fleet of identical GPUs. Multi-tenant sharing, heterogeneous hardware, and prefill/decode disaggregation [Zhong et al., 2024, Patel et al., 2024] are not characterized here and would require additional cost terms. The score uses only HTTP-exposed metrics, not scheduler-internal signals [Pan et al., 2025, Yang et al., 2025a] that could tighten the prefill estimate.

Table 3: **p50-objective TTFT/E2E across all three windows** (ARTChat-411k; seconds). GORGO is `gorgo-hillclimb` in W1 and `gorgo-static` (frozen p50-tuned weights) in W2a/W2b. Bold = best in column within each window. All policies 100% success. `random` was not run in the W2a p50 window and is intentionally absent there.

	TTFT (s)			E2E (s)		
Policy	p50	p95	p99	p50	p95	p99
W1 — tuning window (nighttime)						
GORGO (hillclimb)	**0.163**	**0.943**	2.185	1.45	2.40	3.95
simple-session-affinity	0.196	0.988	**1.654**	1.48	2.61	3.47
least-request	0.207	1.102	1.934	**1.24**	**2.10**	3.07
least-load	0.264	1.079	1.874	1.40	2.38	**2.92**
random	0.274	1.186	2.021	1.27	2.33	3.24
prefix-cache	0.287	1.052	1.773	1.31	2.34	3.32
W2a — held-out (nighttime)						
GORGO (static)	**0.157**	**0.961**	**1.776**	1.42	2.42	3.27
simple-session-affinity	0.206	0.995	1.960	1.56	2.67	3.45
least-request	0.238	1.180	1.812	**1.25**	**2.27**	3.27
least-load	0.289	1.298	2.052	1.41	2.57	3.27
prefix-cache	0.290	1.160	1.944	1.51	2.51	8.65
W2b — held-out (midday diurnal)						
GORGO (static)	**0.188**	**1.184**	**1.901**	1.60	2.92	4.83
simple-session-affinity	0.240	1.498	2.523	1.71	3.78	8.21
random	0.289	1.421	2.331	1.40	2.93	3.97
prefix-cache	0.293	1.451	2.973	1.59	3.22	5.29
least-request	0.293	1.345	2.167	**1.36**	**2.68**	**3.83**
least-load	0.303	1.831	3.144	1.66	3.91	8.36

Table 4: Learned weights under different objective metrics, same tuning trace and hyperparameter ranges. The ES discovers qualitatively different operating points depending on which percentile it optimizes.

Objective	w_{rtt}	w_{prefill}	w_{load}
neg_p95_ttft	0.39	1.88	6.38
neg_p50_ttft	1.21	0.52	1.69

Live-tuning instability. Running the $(1{+}1)$-ES live can produce a heavy-tailed p99 excursion: under heavier midday load `gorgo-hillclimb` p99 inflates to 7.186 s (Table 7) because the negative-p95 objective does not penalize a single bad-minute proposal fast enough. Freezing the learned weights for production deployment is therefore a safety property, not just a convenience: `gorgo-static` with frozen W1 weights avoids this excursion in every held-out window. Decode dynamics such as the effect of variable-length decode and high memory consumption on load may compound live-tuning instability; exploring the impact of higher output token limits would be a direction for future work.

7 Related Work

Prefix caches and reuse. RadixAttention in SGLang [Zheng et al., 2024b] stores per-request KV state in a radix tree; PagedAttention in vLLM [Kwon et al., 2023] enables prefix sharing through paged memory. KVLink [Yang et al., 2025b], ChunkKV [Liu et al., 2025], KVFlow [Pan et al., 2025], and Learned Prefix Caching [Yang et al., 2025a] improve the single-replica cache but do not decide which replica serves a request.

Cross-replica routing and cross-region serving. Preble [Srivatsa et al., 2025] introduces longest-prefix-match routing across replicas with a load-balance fallback; AIBrix [AIBrix Team, 2024] packages a production-grade variant of the same design and supplies the baseline we run. Mooncake [Qin et al., 2024] is a KV-cache-centric architecture and the source of our trace format. SkyServe [Mao

et al., 2025] and SkyWalker [Xia et al., 2025] address cross-region LLM serving at the placement and spillover layers respectively, but treat per-request routing as out of scope. k-LPM [Dexter et al., 2025] formulates LLM scheduling under TTFT constraints as NP-hard. DLPM [Cao et al., 2025] targets fairness with locality. Llumnix [Sun et al., 2024] migrates requests across replicas. Multi-LLM routers [Wu and Silwal, 2025] address model selection. CacheBlend [Yao et al., 2024] routes by trie-measured prefix length but does not measure network RTT. DistServe [Zhong et al., 2024] and Splitwise [Patel et al., 2024] disaggregate prefill from decode. GORGO is, to our knowledge, the first single-router policy in this space that scores cache locality, replica load, and wide-area latency in one cost and derives the cost weights from the deployment's own per-request TTFT stream, with no engine modifications.

Online hyperparameter adaptation. Bayesian and bandit-based methods have been applied to serving configuration [Wu and Silwal, 2025]. For a three-dimensional parameter space the $(1{+}1)$-ES [Rechenberg, 1973] provides fast, overhead-free convergence with no surrogate model. *[Reviewer comment: "2-dimensional" contradicted §3, where the ES tunes three weights $(w_{\mathrm{rtt}}, w_{\mathrm{prefill}}, w_{\mathrm{load}})$.]*

8 Conclusion

Routing across LLM replicas is a three-signal decision balancing cache locality, replica load, and wide-area latency, but production heuristics commit to one signal and degrade when that stops being the limiting factor. We treat the three as terms of an additive per-replica cost and let online TTFT feedback optimize scaling weights, in place of operator-tuned constants or offline profiling. On a long-context, high-prefix-reuse production trace the GORGO policy family achieves the lowest TTFT p50 and p95 in the tuning window and in every held-out evaluation window, and sweeps all reported percentiles under the heavier midday load. On a short-prompt, low-reuse public trace (Appendix A) the same policy is non-competitive, in agreement with the regime characterization in §2. The advantage is therefore a property of the workload regime as much as of the policy: it scales with how much of TTFT is recoverable through prefill reuse, queue avoidance, or network selection rather than fixed by hardware. The design choices of GORGO transfer to deployments where the workload regime is not known in advance and a dedicated calibration window before each redeploy is not affordable, making it effective in production LLM serving on long context, distributed workloads.

References

AIBrix Team. AIBrix: An open-source infrastructure for LLM inference at scale. `https://github.com/vllm-project/aibrix`, 2024.

Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.

Gregory Dexter, Shao Tang, Ata Fatahi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

GLM Team. GLM-4: A family of open multilingual multimodal chat LLMs, 2024.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. SkyServe: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3717459.

Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2507.07400.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yanzhe Wu, Weimin Zheng, Xinran Chen, Rui Xu, Xingda Yao, Rongxin Wei, Zhiyuan Zhao, Jing Huang, Mingjie Yang, and Jidong Wu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.

Qwen Team. Qwen3 technical report. 2025. Alibaba Cloud.

Ingo Rechenberg. Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biologischen evolution. 1973.

Vikranth Srivatsa, Sumanth Madhavan, Tian Hu, Sarah Tarun, Yuhan Cheng, Wei Han, Jingyu Yu, Roberto Cristian Rusti, Lifeng Wang, Yiying Liu, and Hao Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. In *International Conference on Learning Representations (ICLR)*, 2025.

Biao Sun, Ziming Huang, Hanyu Tang, Chengquan Wang, Cheng Zhang, Yiming Li, Liu Yang, Wencong Zhang, Lin-Wei Wang, Tianhe Wu, Ang Cao, Yongqiang Lin, et al. Llumnix: Dynamic scheduling for large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

Fangzhou Wu and Sandeep Silwal. Efficient training-free online routing for high-volume multi-LLM serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. SkyWalker: A locality-aware cross-region load balancer for LLM inference. arXiv preprint arXiv:2505.24095, 2025.

Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Peng, Hao Zhang, Ion Stoica, Kurt Keutzer, and Michael W. Mahoney. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion, 2024.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M ChatGPT interaction logs in the wild. In *International Conference on Learning Representations (ICLR)*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on Learning Representations (ICLR)*, 2024a.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   Answer: [Yes]
   Justification: The abstract claims (i) public datasets underrepresent long-context multi-turn traffic (quantified in §2), (ii) we evaluate routing policies on a production trace (Tables 2 and 3), and (iii) the GORGO family achieves the best TTFT p50 and p95 in all three windows and sweeps every percentile in the midday window. We are explicit in the introduction and §6 that "GORGO" refers to a family, that nighttime p99 margins are at the noise floor, and that under heavier stress load `gorgo-hillclimb`'s p99 inflates due to ES exploration.

2. **Limitations**
   Question: Does the paper discuss the limitations of the work?
   Answer: [Yes]
   Justification: Section 6 lists single-trace generalization, the W2 p99 noise floor, the single-tenant assumption, `gorgo-autotune` instability, hardware homogeneity, the absence of PD-disaggregation, and ES convergence time.

3. **Theory assumptions and proofs**
   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?
   Answer: [N/A]
   Justification: The paper presents an additive cost model and an online optimizer with no formal theorems.

4. **Experimental result reproducibility**
   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?
   Answer: [Yes]
   Justification: Section 4 specifies model, regions, hardware, fleet topology, concurrency, trace windows, and per-window GORGO seeding. Section 3 gives Equation 2, the $(1+1)$-ES configuration ($\sigma_0 = 0.5$, 1/5-rule, hop 16, window 64), and the median-of-rates fit.

5. **Open access to data and code**
   Question: Does the paper provide open access to the data and code?
   Answer: [Yes]
   Justification: The proxy and policy code are available at https://anonymous.4open.science/r/GORGO-D25A. The ARTChat-411k production trace is not redistributable under its terms of service. The public-dataset characterization (LMSYS, WildChat) is fully reproducible with no proprietary inputs.

6. **Experimental setting/details**
   Question: Does the paper specify all the training and test details?
   Answer: [Yes]
   Justification: Section 4 specifies model, hardware, regions, concurrency, trace windows, max input tokens, and per-window seeding for adaptive policies. ES configuration is given in Section 3.

7. **Experiment statistical significance**
   Question: Does the paper report error bars or appropriate statistical significance information?
   Answer: [Yes]
   Justification: Section 6 derives an approximate p99 standard error and shows that p99 margins between the top policies in the held-out windows are inside roughly one standard error in either direction, while the p50/p95 wins are several standard errors clear of zero. *[Reviewer comment: The previous justification cited a `gorgo-hillclimb` vs. `random` comparison that §6 does not make; harmonized with the rewritten noise-floor paragraph.]*

8. **Experiments compute resources**
   Question: Does the paper provide sufficient information on compute resources?
   Answer: [Yes]

Justification: Each policy runs on a dedicated 3-replica fleet of L40S SGLang servers (tensor-parallel 2, 96 GB VRAM per replica). Across the paper's nine 30-minute windows (W1/W2a/W2b under the p95 and p50 objectives, stress W3/W4, and the WildChat replay), roughly 70 fleet-runs of $\sim$30 min each were executed, each fleet comprising six L40S GPUs across three regions. Total GPU-hours: $\approx$210. *[Reviewer comment: The previous count (16 fleet-runs, $\approx$24 GPU-hours) predates the p50-objective, stress, and WildChat experiments; recomputed from the windows actually reported. Please verify against billing/manifests.]*

9. **Code of ethics**
   Question: Does the research conform with the NeurIPS Code of Ethics?
   Answer: [Yes]
   Justification: The work concerns routing infrastructure. The production trace is used under terms that permit research replay; we release only aggregated prefix-trie statistics, not individual requests. No individuals are identifiable from the released aggregates.

10. **Broader impacts**
    Question: Does the paper discuss potential societal impacts?
    Answer: [Yes]
    Justification: Routing improvements reduce the energy and dollar cost of serving LLMs at fixed quality. Lower-cost serving may accelerate deployment in settings where that is socially harmful; the contributions are infrastructure-level and do not affect model capability.

11. **Safeguards**
    Question: Does the paper describe safeguards for responsible release?
    Answer: [N/A]
    Justification: We do not release datasets or models. The released code is a routing proxy and policy library with no inherent dual-use beyond LLM-serving infrastructure.

12. **Licenses**
    Question: Are creators or owners of assets properly credited?
    Answer: [Yes]
    Justification: SGLang, vLLM, AIBrix, LMSYS-Chat-1M, WildChat-4.8M, and Qwen3 are cited and used under their published licenses.

13. **New assets**
    Question: Are new assets introduced in the paper well documented?
    Answer: [Yes]
    Justification: The proxy, policy library, experiment controller, and trace builder are documented in the repository's top-level documentation. Spec and manifest files are stored alongside the controller.

14. **Crowdsourcing and research with human subjects**
    Question: Does the paper involve crowdsourcing or human subjects?
    Answer: [N/A]
    Justification: No human subjects.

15. **IRB approvals**
    Answer: [N/A]
    Justification: No human subjects.

16. **Declaration of LLM usage**
    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the research methodology?
    Answer: [N/A]
    Justification: LLM serving infrastructure is the object of study. Authors used LLMs for routine writing assistance only.

A WildChat replay (regime-sensitivity control)

We replay a 30-minute window of WildChat-4.8M [Zhao et al., 2024] on the same 2×L40S three-region fleet, with the same eight policies as the ARTChat-411k sweeps. WildChat averages 2,925 input tokens per request and 5.3% intra-user prefix reuse—an order of magnitude shorter than ARTChat-411k and roughly a tenth the reuse—putting it well outside the regime characterized in §2.

Table 5: WildChat-4.8M replay: 30-minute window, c=32. TTFT in seconds. Bold is best in column. Compared to the ARTChat-411k results, GORGO variants are not competitive: `gorgo-static` and `gorgo-autotune` are the worst two policies on p95.

Policy	Completed	p50	p95	p99	max	Succ.%
simple-session-affinity	20,447	0.416	**1.025**	**1.712**	7.75	99.6
least-request	20,517	0.480	1.036	1.731	110.45	100.0
random	20,436	**0.416**	1.077	1.796	15.82	99.6
prefix-cache	20,504	0.497	1.186	2.588	10.63	99.9
least-load	20,484	0.532	1.217	2.535	8.28	99.8
gorgo-hillclimb	20,473	0.541	1.325	2.654	7.05	99.8
gorgo-static	20,462	0.709	1.932	3.969	60.18	99.7
gorgo-autotune	20,451	0.541	3.583	6.438	12.55	99.7

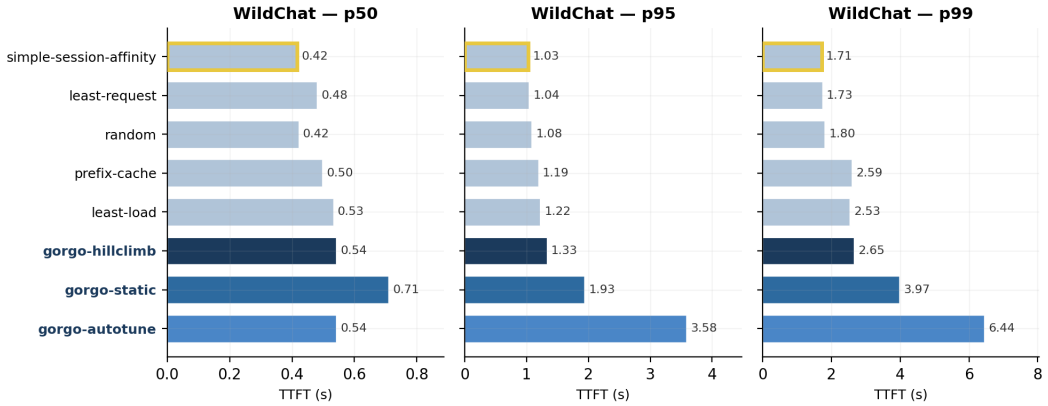

Figure 6: WildChat-4.8M replay, per-percentile TTFT (p50/p95/p99) across the eight policies. GORGO variants (dark blues) are not competitive: `gorgo-static` and `gorgo-autotune` are the slowest two policies at the tail, and `gorgo-hillclimb` sits mid-pack. The ranking inverts the ARTChat-411k result, consistent with the regime argument in §2.

The ranking is reversed: `simple-session-affinity` wins p95 and p99 by a large margin, `least-request` is competitive, `gorgo-hillclimb` sits mid-pack, and `gorgo-static` and `gorgo-autotune` are the worst two policies. Two mechanisms drive this. First, the prefill term in Equation 2 is small at 2,925 tokens; the cost-model weights are mostly fitting noise. Second, intra-user reuse is 5.3% rather than 53%; even a perfect cache-routing decision saves only a few hundred tokens per request, well inside the cross-region RTT band, so chasing cache locality is counterproductive when paired with a cross-region routing penalty. The `gorgo-static` weights frozen from a ARTChat-411k W1 are particularly mis-calibrated to this regime—a reminder that the cost-model parameters do not transfer across workload classes.

We include this result as a regime-sensitivity control, not as a contribution. The takeaway is the same one stated in §2: the gain from joint cache–network–queue routing depends on the workload having long prompts and substantial intra-user prefix reuse. Public chat datasets do not exercise this regime.

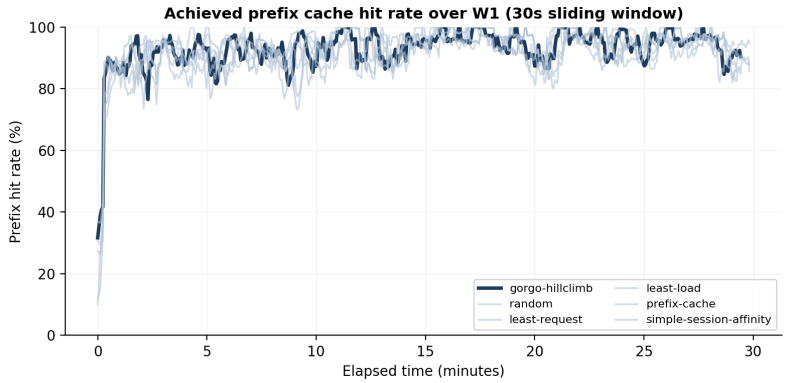

Figure 7: Achieved prefix hit rate over the W1 tuning window: the fraction of each request's input tokens that were already cached on the replica the policy routed to (not the total cache available across all replicas). A higher rate means the routing decision exploited more of the available cache. Bold lines are EWMA-smoothed (α=0.06); faint lines are 64-request rolling averages. `gorgo-hillclimb` converges from ∼45% to ∼95% within the first 5 minutes as the ES learns to route requests to their best-cached replica.

B GORGO Achieves Convergence of Cache Reuse

C Load-Weight Ablation: Single-Replica Concentration

When the search range permits a near-zero load weight ($w_{\text{load}} \in [10^{-5}, 5]$), the ES drives it to zero: the resulting policy routes 100% of traffic to a single replica (Figure 8). On the nighttime tuning trace (∼1.2 req/s) this produces the best TTFT because every request hits a warm cache on the closest replica with no load penalty. On the heavier midday diurnal trace (∼1.7 req/s, 47% more requests), the single replica saturates: decode throughput drops to 70 tok/s (vs. 119 tok/s with w_{load}=6.38) and E2E p95 inflates to 12.58 s—4.8× worse than `least-request`.

Constraining the ES search range to $w_{\text{load}} \in [0.1, 10.0]$ prevents this degenerate solution. *[Reviewer comment: The two ranges differ at both ends; the operative change is the lower bound (0.1 vs. 10^{-5}). Either use the same upper bound in both runs or note that the upper bound is not binding, so the comparison isolates the lower-bound effect.]* The ES converges to w_{load}≈6.38, which actively avoids loaded replicas and achieves the best TTFT *and* E2E on both evaluation windows (§5.1). Table 6 compares the two configurations on the midday diurnal trace.

Table 6: Midday diurnal evaluation: unconstrained (w_{load}=0) vs. constrained (w_{load}=6.38) `gorgo-static`. Constraining w_{load} trades 3% TTFT p95 for 5× better E2E p95.

	TTFT p50	TTFT p95	E2E p50	E2E p95	Decode	Concentration
w_{load}=0	**0.190**	**1.101**	2.07	12.58	70	100%
w_{load}=6.38	0.195	1.136	**1.29**	**2.46**	**119**	60%
Δ	+2.6%	+3.2%	−37.7%	−80.4%	+70%	

D Stress-Load Robustness

To test robustness under sustained high load, we ran all eight policies on two additional 30-minute windows at concurrency c=64: a midday window (Apr 1, 12:30–13:00 UTC, ∼1.7 req/s, "W3") and a night window (Apr 2, 00:30–01:00 UTC, ∼1.5 req/s, "W4"). W4 covers the same wall-clock period as the main W1 tuning window but is an independent experiment run at higher effective load (denser arrival bursts); it shares neither caches nor routing state with W1. *[Reviewer comment: "Higher effective load (denser arrival bursts)" over the same wall-clock trace needs quantification: state what was changed relative to W1 (replay speed-up factor? burst injection?) and the resulting*

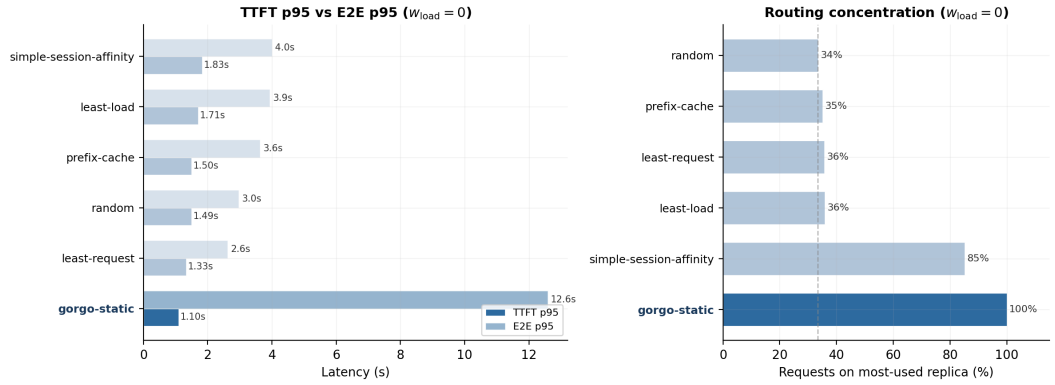

Figure 8: *Left*: TTFT p95 (dark) vs. E2E p95 (light) on the midday diurnal trace with w_{load}=0. `gorgo-static` wins TTFT but its E2E inflates to 12.6 s. *Right*: routing concentration. `gorgo-static` sends 100% of requests to one replica.

req/s. Without this, W4 is not reproducible and the W1-vs-W4 success-rate gap (100% vs. ∼89.5% at the same c=64) is unexplained.] Both windows use the same frozen weights as W2 (`gorgo-static`: W1 hillclimb output; `gorgo-hillclimb`: ES continues live). Success rate drops to ∼89.6% under stress as some requests time out.

Table 7: W3 (stress midday, Apr 1 12:30–13:00 UTC, c=64). TTFT in seconds. Bold is best in column. `gorgo-static` wins p95; `gorgo-hillclimb` produces the worst p99 (7.186 s) due to ES exploration under load.

Policy	OK	Succ.%	p50	p95	p99	max	E2E p95	E2E p99
gorgo-static	3,372	89.6	**0.193**	**1.400**	2.140	3.37	**3.491**	13.830
prefix-cache	3,376	89.7	0.295	1.416	**1.973**	3.75	3.794	13.221
simple-session-affinity	3,375	89.7	0.324	1.531	2.487	28.07	4.047	15.537
gorgo-hillclimb	3,376	89.7	0.226	1.581	7.186	38.29	4.283	16.790
least-request	3,380	89.8	0.275	1.694	2.440	20.07	3.896	13.162
random	3,371	89.6	0.309	1.700	2.421	3.99	4.445	15.073
least-load	3,374	89.7	0.309	1.735	2.419	16.91	4.596	15.263
gorgo-autotune	3,377	89.7	0.299	1.748	2.519	23.63	4.519	17.971

Table 8: W4 (stress night, Apr 2 00:30–01:00 UTC, c=64). TTFT in seconds. Bold is best in column. `gorgo-hillclimb` wins p95 and p99; `gorgo-static` achieves the lowest p50. `simple-session-affinity` was not included in this window.

Policy	OK	Succ.%	p50	p95	p99	max	E2E p95	E2E p99
gorgo-hillclimb	2,466	89.5	0.281	**1.331**	**1.832**	3.05	**3.287**	**11.156**
prefix-cache	2,468	89.6	0.288	1.394	1.992	2.95	3.585	13.025
gorgo-static	2,472	89.8	**0.179**	1.427	2.302	4.06	3.340	10.463
least-request	2,465	89.5	0.281	1.605	2.268	4.16	3.325	11.240
gorgo-autotune	2,469	89.7	0.280	1.690	2.298	18.84	3.441	13.580
least-load	2,465	89.5	0.281	1.700	2.290	2.98	3.659	13.100
random	2,460	89.3	0.287	1.724	2.357	4.00	3.827	12.781

The stress windows confirm the main findings with an important safety note: under heavier load (W3), `gorgo-hillclimb`'s live ES exploration generates the worst p99 in the field (7.186 s), a 3.6× excursion over the best p99 in the field (`prefix-cache`, 1.973 s) and 2.9× the next-worst policy (`gorgo-autotune`, 2.519 s). *[Reviewer comment: The previous text attributed the 3.6× ratio to the next-worst policy; per Table 7 that ratio is 2.9×, while 3.6× is the ratio to the best.]* This is the primary motivation for the production recommendation to freeze learned weights (`gorgo-static`) rather than run the ES live. `gorgo-static` wins W3 on both TTFT p95 and E2E p95.

Under the lower-load stress night (W4), `gorgo-hillclimb` recovers to win p95 (1.331 s vs. `gorgo-static`'s 1.427 s), consistent with the main W1 tuning-window result: live ES works well when load is low enough that exploration proposals carry little tail risk. `gorgo-static`'s p50 (0.179 s) is the lowest in the field in W4, reflecting the quality of the frozen W1 weights even under shifted arrival rates.